

10일 차

오토인코더 비지도학습 기초

이번 차시에서 주목할 키워드

- 비지도학습
- 오토인코더
- 잠재 공간
- 잠재 벡터
- 차원 축소

10일 차 학습 목표

- 비지도학습의 개념과 정답이 없는 데이터를 활용하는 이유를 설명할 수 있습니다.
- 오토인코더의 구조와 원리, 그리고 특징을 스스로 학습하는 과정을 설명할 수 있습니다.
 - MNIST 노이즈 제거 예제로 오토인코더 모델을 직접 설계하고 학습하며, 입출력의 차이를 비교해 복원 성능을 분석합니다.

지금까지 이 책에서는 주로 정답이 있는 데이터를 학습하는 지도학습을 다루었습니다. 하지만 실제 데이터의 대부분은 정답이 없는 경우가 많습니다. 따라서 목표 값이 없는 데이터를 이해하고 활용하는 비지도학습 또한 딥러닝에서 매우 중요한 분야입니다. 마지막 차시에서는 비지도학습의 개념을 소개하고 대표적인 기법 중 하나인 오토인코더(Autoencoder)를 소개합니다. 오토인코더의 구조를 이해하고 이미지 노이즈의 제거와 차원 축소 예제로 비지도학습의 원리를 익혀 보겠습니다.

비지도학습 개요

비지도학습이란?

이 책에서는 지금까지 딥러닝에서 ‘정답’이 있는 데이터를 학습하는 방법, 즉 지도 학습(Supervised Learning)을 설명했습니다. 지도학습은 고양이와 강아지를 구분하는 이미지 분류, 텍스트로 된 스팸 메일 분류처럼 ‘입력’과 함께 언제나 ‘정답(Label)’이 주어진 상황에서 학습하는 모델입니다.

하지만 현실에서는 데이터에 정답이 달려 있지 않은 경우가 많습니다. 예컨대 수많은 고객의 행동 데이터, 웹에서 수집된 이미지, 의료 센서 데이터 등에는 별도의 정답이 붙어 있지 않습니다. 그렇다고 이런 데이터가 쓸모가 없을까요? 절대 그렇지 않습니다. 오히려 정답이 없는 데이터에서 의미 있는 패턴을 찾는 일이 더욱 중요해지고 있습니다. 그리고 이 점이 비지도학습(Unsupervised Learning)의 목표입니다.

지도학습과 비지도학습의 차이

[표 10-1]에서는 지도학습과 비지도학습의 차이를 표로 정리했습니다.

[표 10-1] 지도학습과 비지도학습 비교

구분	지도학습(Supervised)	비지도학습(Unsupervised)
데이터 구성	입력+목표값(레이블)	입력만 존재(목표값 없음)
목표	목표값을 예측하는 모델 학습	데이터 내부의 패턴, 구조 파악
예시	이미지 분류, 가격 예측	클러스터링, 차원 축소, 이상 탐지 등

비지도학습은 목표값이 없기 때문에 모델이 '무엇을 맞춰야 하는지'가 명확하지 않습니다. 대신 모델은 데이터를 스스로 분석하며 '서로 유사한 데이터는 가까이, 다른 데이터는 멀리' 두는 방식으로 데이터의 구조를 파악합니다.

비지도학습이 필요한 이유

비지도학습은 다음과 같은 상황에서 특히 유용합니다.

- 정답(Label)이 없는 대규모 데이터 세트: 실제 현장에서 수집되는 데이터는 많지만, 정답이 있는 경우는 드뭅니다. 예를 들어 쇼핑물의 구매 로그나 CCTV 영상은 매일매일 방대한 양의 정보가 쌓이지만, 하나하나 레이블링(Labeling)하기에는 시간과 비용이 많이 들어 사실상 불가능합니다. 이때 비지도학습은 정답이 없어도 데이터를 분석하고 활용할 수 있는 좋은 방법입니다.
- 데이터 구조를 파악하고 싶을 때: 고차원 데이터는 눈으로 보기 어려울 만큼 복잡합니다. 이때 비지도학습을 이용하면 데이터를 저차원으로 줄여 시각화하거나 주요 특성을 요약해 볼 수 있어 유용합니다. 예를 들어 수천 개의 상품 속성을 몇 개의 대표 특성으로 압축해 분석하면 데이터의 전반적인 구조를 직관적으로 이해할 수 있습니다.
- 이상 탐지(Anomaly Detection): 정상적인 패턴을 학습하면 정상 패턴에서 벗어난 특이한 데이터는 자동으로 찾을 수 있습니다. 예를 들어 신용카드 결제 내역에서 평소와 다른 의심스러운 거래를 감지하거나 기계 센서 데이터의 고장을 예측할 때도 비지도학습이 유용하게 활용될 수 있습니다.

비지도학습은 정답이 없는 데이터에서 의미 있는 패턴과 구조를 찾는 강력한 도구이므로 점점 더 다양한 비즈니스 상황에서 많이 활용되고 있습니다.

대표적인 비지도학습 기법

비지도학습은 크게 두 가지 방향으로 발전해 왔습니다. 바로 클러스터링(Clustering)과 차원 축소(Dimensionality Reduction)입니다. 두 방법은 모두 정답이 없는 데이터에서 숨겨진 패턴이나 구조를 찾는 데 활용되지만, 접근 방식과 목적이 서로 다릅니다.

클러스터링

클러스터링은 이름 그대로 비슷한 데이터를 묶어 하나의 그룹(Cluster)을 만드는 방법입니다. 마치 학생들이 같은 취미나 관심사를 가진 친구들과끼리 모여 동아리를 결성하는 일과 비슷합니다. 클러스터링은 데이터 사이의 유사도(얼마나 닮았는가)를 기준으로 이루어지는데, 정답이 없어도 자동으로 그룹이 만들어진다는 장점이 있습니다.

대표적인 클러스터링 알고리즘은 다음과 같습니다.

- K-means 클러스터링: 사용자가 미리 그룹의 개수를 정하면 알고리즘이 각각의 데이터를 가장 가까운 그룹에 배정하면서 그룹을 형성합니다. 예를 들어 고객을 3개의 그룹으로 나눈다고 하면 '가격에 민감한 그룹', '프리미엄 선호 그룹', '보통 소비 그룹'과 같이 세 유형으로 분류할 수 있습니다.
- 계층적(Hierarchical) 클러스터링: 데이터를 점차 합치거나 쪼개면서 나무(Tree) 형태의 구조로 그룹을 만듭니다. 작은 그룹이 모여 큰 그룹을 형성하는 방식이어서 데이터 사이의 관계를 계층적으로 살펴볼 때 유용합니다.
- DBSCAN: 데이터의 밀도를 기준으로 그룹을 형성합니다. 데이터가 뽕뽕하게 모여 있는 부분은 하나의 그룹으로 묶고 떨어져 있는 데이터는 '이상치(Outlier)'로 판별할 수 있습니다. 복잡한 형태의 데이터에서도 잘 작동합니다.

이와 같은 클러스터링 기법은 비즈니스에서 다양하게 응용할 수 있습니다. 예를 들어 온라인 쇼핑몰의 고객 데이터를 클러스터링하면 구매 패턴이 유사한 고객 집단을 손쉽게 찾을 수 있습니다. 구매 패턴을 기반으로 '할인을 좋아하는 고객 그룹'에는 쿠폰을 제공하고, '신제품을 선호하는 그룹'에는 신상품 정보를 먼저 알리는 등 맞춤형 마케팅 전략을 세울 수 있습니다.

차원 축소

차원 축소는 고차원 데이터를 더 단순한 저차원으로 줄이는 과정입니다. 쉽게 말해 변수가 많아 복잡하게 얽힌 데이터를 핵심만 뽑아 요약하는 방법입니다. 예컨대 수십 가지 특성을 가진 고객 데이터를 '구매력'과 '브랜드 충성도' 두 개의 축으로만 표현한다면 훨씬 직관적으로 이해할 수 있습니다.

대표적인 차원 축소 기법은 다음과 같습니다.

- PCA(Principal Component Analysis, 주성분 분석): 데이터를 가장 잘 설명하는 직각 방향(주성분)을 찾아 차원을 줄이는 방법입니다. 예를 들어 3차원 데이터가 있다면 그중 가장 정보가 많은 두 방향만 남겨 2차원 평면에 표현합니다. 선형(직선) 기반이라 빠르고 직관적이지만, 복잡한 곡선 구조는 잘 표현하지 못합니다.
- t-SNE: 데이터 사이의 유사도를 최대한 보존하면서 2D나 3D 공간을 시각화하는데 특화된 기법입니다. 특히 클러스터링 구조를 시각적으로 보여주는 데 강점이 있어 고차원 데이터를 분석할 때 자주 활용됩니다. 다만 계산 속도가 느리다는 단점이 있습니다.
- UMAP: t-SNE와 비슷하게 고차원 데이터를 저차원으로 시각화하지만, 더 빠르며 큰 데이터 세트에서도 잘 작동합니다. 또한 클러스터링의 전반적인 구조를 더 잘 보존한다는 장점이 있습니다. 최근 데이터 시각화에 많이 활용되는 기법입니다.

이번 차시에서 다룰 오토인코더 역시 강력한 차원 축소 도구입니다. 신경망을 이용해 데이터를 잠재 공간(Latent Space)으로 압축하고 다시 복원하는 과정에서 데이터의 중요한 특징만 남기고 불필요한 정보를 버립니다. 차원 축소 기법을 활용하면 복잡한 데이터를 효율적으로 다룰 수 있습니다. 즉, 차원 축소는 데이터를 2D 또는 3D로 줄여 시각화할 수 있고, 데이터의 노이즈를 제거하거나 요약된 특성을 다른 학습에 활용하는 데도 유용합니다.

정리하면 클러스터링은 비슷한 데이터를 묶어 그룹을 만드는 작업이고, 차원 축소는 데이터를 더 간단하게 줄여 표현하는 작업입니다. 둘 다 비지도학습의 핵심 기법으로서 이들 기법을 잘 활용하면 정답이 없는 데이터에서도 충분히 의미 있는 정보를 얻을 수 있습니다.

비지도학습과 딥러닝의 만남

과거의 비지도학습은 주로 통계적 기법이나 규칙 기반 알고리즘에 의존했습니다. 이 방법들은 단순한 데이터에서는 효과적이지만, 복잡한 이미지나 텍스트 같은 고차원 데이터에서는 한계가 뚜렷했습니다. 최근 들어 딥러닝이 비지도학습에 접목되면서 상황은 크게 달라졌습니다.

중심에는 오토인코더(Autoencoder)라는 모델이 있습니다. 오토인코더는 주어진 데이터를 스스로 압축하고 다시 복원하는 과정을 반복하면서 데이터에 숨어 있는 중요한 특징을 자동으로 학습합니다. 예를 들어 이미지라면 선과 모양, 텍스트라면 단어 사이의 의미 관계 같은 본질적인 구조를 오토인코더는 스스로 파악합니다.

오토인코더는 사람이 일일이 특징을 정의하지 않아도 모델이 데이터의 핵심 표현(Representation)을 학습할 수 있도록 해줍니다. 오토인코더는 단순한 복원을 넘어 데이터의 본질적인 패턴을 포착해 새로운 활용 가능성을 열어 주는 도구입니다.

이어지는 절들에서는 오토인코더의 기본 작동 원리와 구조를 살펴보고 실습으로 구현하면서 그 원리를 이해해 보겠습니다.

오토인코더의 개념과 구조, 활용

오토인코더란?

혹시 위폐 감별사가 위조지폐를 어떻게 구별하는지 아시나요? 감별사는 수많은 위조지폐를 분석하지 않습니다. 오히려 진짜 지폐만을 반복해 관찰하고 그 속에 담긴 특징들을 철저히 익힙니다. 홀로그램의 위치, 미세한 문자, 숨은 은선, 색상의 변화 같은 진짜 지폐만이 가진 미묘한 요소들을 감별사는 머릿속에 정확히 새겨 두는 것이죠. 즉, 진짜의 특징을 명확히 알고 있다면 그와 다른 점을 기준으로 위조 여부를 쉽게 판단할 수 있습니다.

이와 마찬가지로 오토인코더도 목뽕값 없이 진짜 데이터만 보고 중요한 특징을 학습합니다. 오토인코더는 정답을 따로 알려 주지 않기 때문에 오직 진짜 데이터만 보고 학습합니다. 고양이 사진, 손글씨 이미지처럼 실제 데이터를 반복해 보여 주면 모델은 그 안에 담긴 중요한 특징을 스스로 파악합니다. 그리고 학습한 특징으로 입력 데이터를 복원하려고 합니다.

예를 들어 숫자 '8'의 이미지를 여러 번 학습한 오토인코더는 '8'이 어떤 형태인지

자연스럽게 이해합니다. 이후 이상한 모양의 숫자, 예를 들어 선이 하나 빠지거나 비정상적으로 찌그러진 이미지를 입력으로 받으면 기존에 학습한 특징과 다르므로 제대로 복원하지 못합니다. 복원 결과가 흐릿하거나 이상하다면 오토인코더는 ‘무언가 이상하다’는 신호를 내보냅니다.

오토인코더는 진짜 데이터를 이용해 중요한 특징을 압축 학습하는데, 학습한 정보만으로 복원과 이상 탐지까지 모두 가능한 강력한 비지도학습 딥러닝 모델입니다. 한마디로 말해 입력을 압축했다가 다시 복원하는 형태의 신경망입니다. 예를 들어 ‘고양이 사진’을 입력으로 넣으면 오토인코더 모델은 그 사진의 핵심적인 정보를 압축해서 기억한 다음, 원래의 고양이 사진처럼 복원하려고 합니다. 입력과 출력이 거의 동일한 데이터가 되도록 학습한 신경망이기 때문입니다.

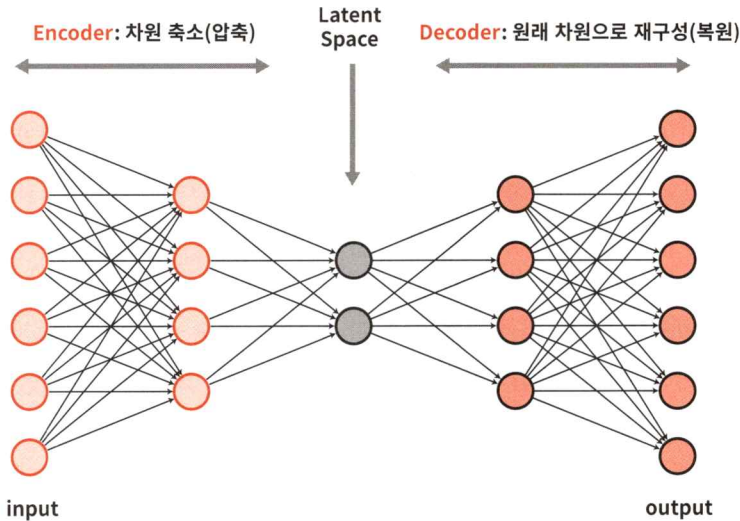
그렇다면 오토인코더는 왜 이런 일을 할까요? 단순히 사진을 복제하려는 게 아닙니다. 오토인코더는 ‘복원하는 과정’ 그 자체로 데이터의 핵심적인 특징을 파악하려는 것이 목적입니다. 예를 들면 사람이 책을 요약한 다음, 요약을 다시 떠올려 원래 내용을 복원하려는 행동과 비슷합니다. 다시 복원한다고 했을 때 요약은 어떤 형태일까요? 당연히 중요 정보만 남기고 불필요한 정보는 제외한 형태일 겁니다.

또 다른 예를 들어 볼까요? 어떤 사람이 ‘파란 배경에 검은 고양이가 앉아 있는 사진’을 본다고 합시다. 이 사람에게 그 사진을 요약하라고 하면 “고양이가 파란 배경 위에 앉아 있었어”라고 말할 가능성이 높습니다. 고양이의 눈썹 길이나 사진 구도 같은 세세한 정보는 생략하고 말입니다. 오토인코더는 입력을 압축하면서도 중요한 특징은 유지하는 학습을 합니다. 이 과정의 핵심 요소가 바로 ‘잠재 표현(Latent Representation)’ 또는 ‘잠재 공간(Latent Space)’입니다. 오토인코더는 이 잠재 공간에 입력 데이터를 압축해 표현하는데, 이 공간은 입력보다 훨씬 낮은 차원의 벡터로 구성되어 있습니다. 이 잠재 공간은 모델이 학습한 데이터를 압축한 ‘요약 정보’라고 할 수 있습니다.

오토인코더는 비단 이미지뿐만 아니라 텍스트, 음성, 센서 데이터 등 다양한 형태의 데이터에 적용할 수 있어 차원 축소, 노이즈 제거, 이상 탐지 등 여러 작업에서 응용할 수 있습니다.

오토인코더의 구조와 학습 방식

오토인코더의 구조는 크게 두 부분으로 나뉩니다. 바로 인코더(Encoder)와 디코더(Decoder)입니다. 두 구성 요소가 입력 데이터를 압축하고 다시 복원하는 역할을 합니다.



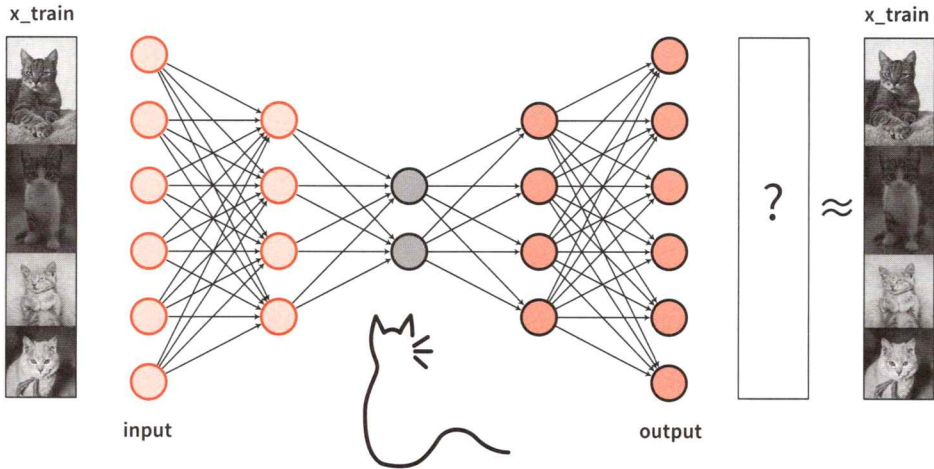
[그림 10-1] 오토인코더의 구조

인코더는 입력 데이터를 점점 줄여 가며 압축하는 역할을 수행합니다. 이 과정에서 입력 데이터는 잠재 공간(Latent Space)이라고 불리는 곳에서 압축된 중간 결과물로 변환됩니다. 잠재 공간은 일반적으로 입력보다 훨씬 작은 차원으로 구성되는데, 인코더로 생성된 중간 결과물은 ‘데이터의 핵심 요약본’이라고 할 수 있습니다. 잠재 공간에 있는 값을 ‘잠재 벡터’라고 합니다.

디코더는 압축된 잠재 벡터를 다시 원래의 데이터로 복원하는 역할을 수행합니다. 즉, 인코더가 ‘요약’이라면 디코더는 ‘되살리기’입니다. 이 두 단계가 합쳐져 하나의 오토인코더를 이루며 전체 구조는 [그림 10-1]처럼 입력을 점차 압축했다가 다시 복원하는 ‘모래시계’ 모습을 띠고 있습니다.

고양이 사진 여러 장을 오토인코더에 학습시킨다고 가정합니다. 모델은 각각의 고양이 사진을 인코더로 압축하고 다시 디코더로 복원하는 과정을 반복하면서 학습합니다. 이때 중요한 것은 입력(Input)과 출력(Output)이 최대한 비슷해지도록 학습한다는 점입니다. 즉, 고양이 사진을 입력했으면 출력도 고양이 사진이 되도록 만들어야 합니다.

복원 과정에서 발생하는 차이(입력과 출력의 차이), 즉 오차를 줄이기 위해 학습된 모델은 고양이의 중요한 특징(예를 들면 귀의 모양, 눈의 위치, 털 패턴)을 점점 잘 기억합니다. 학습이 잘 이루어지면 모델은 고양이 사진을 볼 때마다 공통 특징만으로 다시 비슷하게 그릴 수 있습니다. 오토인코더는 공통의 중요 특징만 남기고 불필요한 정보(예: 배경, 조명, 노이즈 등)는 제거하는 효과가 있습니다.



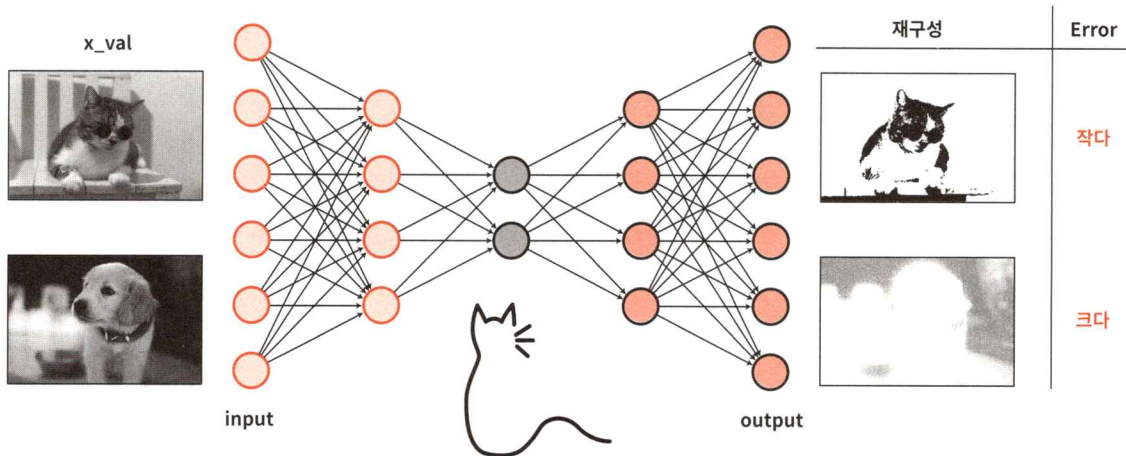
[그림 10-2] 고양이 이미지로 오토인코더 학습

학습을 모두 마친 모델은 '새로운 데이터'로 복원 결과를 비교할 수 있습니다. 예를 들어 그동안 학습한 적이 없는 사진(예를 들면 선글라스를 낀 고양이나 강아지 사진)을 입력하면 어떻게 될까요?

고양이의 특징만 학습한 오토인코더는 익숙한 고양이의 특징은 어느 정도 복원할 수 있지만, 강아지 사진처럼 생소한 데이터는 제대로 복원하지 못합니다. 이때 복원 결과와 원래의 입력 데이터 사이에 차이가 크게 나타나는데, 이를 재구성 오차(Reconstruction Error)라고 합니다.

- 재구성 오차가 작다 → 학습 데이터와 유사하다(예: 고양이)
- 재구성 오차가 크다 → 학습 데이터와 다르다(예: 강아지)

오토인코더는 이런 식으로 이상 탐지나 노이즈 제거 작업 등에 자연스럽게 활용됩니다.



[그림 10-3] 학습된 오토인코더 모델 사용(예측)

정리하면 오토인코더는 압축(Encoder)과 복원(Decoder)으로 구성됩니다. 그리고 입력과 출력이 유사하도록 학습하면서 중요한 특징을 기억합니다. 또한 학습이 잘 된 모델은 이전에는 본 적이 없는 데이터가 들어오면 복원에 실패하는데, 그 차이로 이상을 감지할 수 있습니다.

이어지는 절에서는 오토인코더의 개념과 구조에 대한 이해를 바탕으로 노이즈를 제거하는 비지도학습 모델을 직접 구현하겠습니다.

오토인코더 구현 1: MNIST 노이즈 제거

실습 개요

앞서 배운 오토인코더의 개념과 구조를 실습으로 구현합니다. 실습에 사용할 데이터는 5, 8일 차에서 다뤘던 MNIST 손글씨 이미지 데이터 세트입니다.

이번 실습의 목표는 노이즈가 포함된 이미지에서 원래의 이미지를 복원하는 오토인코더 모델을 직접 구성하는 것입니다. 이 과정에서 인코더와 디코더 구조를 학습해 입력 이미지와 최대한 유사한 출력을 생성할 예정입니다. 또한 복원 결과를 눈으로 직접 확인하면서 오토인코더가 어떻게 데이터의 '본질적인 특징'을 학습하는지 이해합니다.

실습을 마치면 오토인코더가 단순히 원래의 데이터를 복원하는 것에 그치는 게 아니라 데이터의 본질적인 특징을 학습한다는 사실을 확인할 수 있을 겁니다. 이 경험은 다음 절에서 소개할 오토인코더 확장 모델을 이해하는 데 많은 도움이 됩니다.

환경 준비에서 데이터 전처리까지

새 코드 파일을 열고 파일 이름은 '10일차_비지도학습모델링.ipynb'로 합니다. 런타임 유형은 'T4 GPU'를 선택합니다.

환경 준비

이어 라이브러리를 불러오고 디바이스를 준비합니다.

```
CODE
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torchvision
from torchvision.transforms import ToTensor
from torchvision.datasets import MNIST
from torch.utils.data import DataLoader, Dataset
from torch.optim import Adam

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

꼭 필요한 라이브러리만 불러왔으며 디바이스 설정은 종전과 동일합니다. 이전 차시들에서 생성했던 `train`, `evaluate` 같은 사용자 정의 함수를 오토인코더에 맞게 새로 수정할 수도 있지만, 함수의 구조가 달라지면 오히려 혼란을 줄 우려가 있습니다. 따라서 이번 차시에서는 별도의 함수를 생성하지 않고 학습과 평가 과정을 코드로 직접 구현할 예정입니다.

데이터 전처리

MNIST 데이터를 다운로드하고 데이터 로더를 선언합니다. 기존에는 학습 데이터만 데이터 로더로 선언하고 검증 데이터나 테스트 데이터는 텐서 데이터로 준비했지만, 여기서는 코드를 간결하게 작성하기 위해 테스트 데이터 역시 데이터 로더로 선언합니다.

```
CODE
# 다운로드
train_set = MNIST(root='./data', train=True, transform=ToTensor(), download=True)
test_set = MNIST(root='./data', train=False, transform=ToTensor(), download=True)

# 데이터 로더 선언
```

```
train_loader = DataLoader(train_set, batch_size=128, shuffle=True)
test_loader = DataLoader(test_set, batch_size=128, shuffle=False) ①
```

| ① 테스트 데이터도 데이터 로더로 선언합니다.

테스트 데이터는 원래 데이터 로더로 선언할 필요가 없습니다. 하지만 테스트 데이터가 대규모여서 메모리를 효율적으로 관리할 필요가 있는 경우 또는 지금처럼 간결한 코드를 작성하기 위해 테스트 데이터를 데이터 로더로 선언하는 경우가 종종 있습니다.

테스트 데이터를 데이터 로더로 선언할 때 데이터를 섞지 않은(`shuffle=False`) 이유는 테스트 데이터는 섞을 필요가 없다는 관례에 따른 것인데, 섞으나 안 섞으나 테스트 데이터 전체로 예측하고 평가하므로 결과에는 영향을 주지 않습니다.

데이터에 노이즈 추가하기

이제 각각의 손글씨 이미지에 정규분포에서 추출한 노이즈(잡음)를 추가합니다. 오토인코더 모델로 추가한 노이즈를 제거하고 원본 이미지를 복원하는 작업이 이번 실습의 목표입니다.

다음과 같이 노이즈를 추가하는 함수를 작성합니다.

CODE

```
def add_noise(imgs, noise_factor=0.5):
    noisy_imgs = imgs + noise_factor * torch.randn_like(imgs) ①
    noisy_imgs = torch.clip(noisy_imgs, 0., 1.) ②
    return noisy_imgs
```

① `torch.randn_like(imgs)`는 입력 이미지와 같은 크기이면서 표준 정규분포(평균 0, 표준편차 1)를 따르는 무작위 값을 생성합니다. 여기에 `noise_factor`를 곱해 노이즈의 크기를 조절합니다. 그리고 원본 이미지 `imgs`에 이 노이즈를 더해 `noisy_imgs`를 만듭니다.

② MNIST 같은 이미지 데이터는 보통 픽셀 값이 0~1 사이입니다. 그러나 노이즈를 추가하면 값이 0보다 작아지거나 1보다 커질 수 있습니다. `clip` 메서드를 사용해 값이 0~1 범위를 벗어나지 않도록 잘라 줍니다.

노이즈를 추가하는 함수에 대해 몇 가지 부연 설명을 하겠습니다. 먼저 표준 정규분포(평균 0, 표준편차 1)를 따르는 무작위 값을 사용하는 이유는 노이즈가 특정 방향으로 치우치지 않고 고르게 분포하도록 하기 위함입니다. 이렇게 하면 어떤 픽셀은 밝아지고 어떤 픽셀은 어두워지면서 보다 자연스러운 노이즈를 만들 수 있습니다.

다. 또한 clip 메서드를 이용해 값이 0~1 범위를 벗어나지 않도록 잘라 주면 노이즈를 추가하더라도 픽셀 값이 허용된 범위 안에서 유지됩니다. 이렇게 하면 데이터는 깨지지 않고 안정적으로 학습에 활용할 수 있습니다.

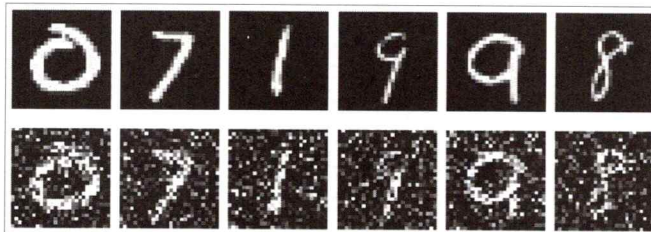
작성한 함수를 이용해 노이즈를 추가하고 몇 개의 이미지를 출력합니다. 빈 셀에서 다음과 같이 코드를 작성합니다.

```
CODE
x, _ = next(iter(train_loader)) ①
noisy_x = add_noise(x) ②

plt.figure(figsize=(10, 4)) ③
for i in range(6): ④
    # 원본 이미지
    plt.subplot(2, 6, i + 1) ⑤
    plt.imshow(x[i][0], cmap='gray') ⑥
    plt.axis('off') ⑦
    # 노이즈 이미지
    plt.subplot(2, 6, i + 7)
    plt.imshow(noisy_x[i][0], cmap='gray') ⑧
    plt.axis('off')

plt.tight_layout() ⑨
plt.show()
```

OUTPUT



- ① next와 iter 함수를 중첩해 실행하면 train_loader에서 첫 번째 배치를 꺼냅니다. 학습 이미지 텐서만 사용할 예정이므로 정답은 _로 버립니다.
- ② 정의한 add_noise 함수를 적용해 x에 노이즈를 추가합니다. 픽셀 값은 0~1 범위로 클리핑됩니다.
- ③ 이미지의 크기를 가로 10, 세로 4인치로 지정합니다.
- ④ 배치에서 앞의 6개 샘플만 시각화하기 위해 0부터 5까지만 반복합니다.
- ⑤ 2행 6열 격자 중 뒷줄 i+1번째 위치에 서브 플롯을 만듭니다. 뒷줄에는 원본 이미지를 배치합니다.
- ⑥ x[i][0]: 데이터 세트 x에서 i번째 이미지를 꺼내면 구조(shape)는 [1, 28, 28]입니다. 여기서 [0]을 한 번 더 붙이면 [28, 28] 크기가 됩니다. 이 이미지를 회색조의 컬러 맵

(cmap='gray')으로 화면에 출력합니다.

- ⑦ 눈금과 축을 감춰 이미지에만 집중할 수 있게 합니다.
- ⑧ i번째 노이즈 이미지를 회색조로 표시합니다.
- ⑨ 서브 플롯 사이의 여백을 자동으로 조정해 제목이나 이미지가 겹치지 않도록 합니다.

학습 데이터에서 6개의 이미지를 추출하고 각각의 이미지에 노이즈를 추가한 다음, 원본 이미지와 노이즈가 추가된 이미지를 나란히 출력했습니다. 코드의 실행 결과를 보면 제일 오른쪽에 있는 숫자 8의 이미지는 사람이 알아보기 힘들 정도의 노이즈를 포함하고 있습니다.

오토인코더 모델 학습 및 예측

오토인코더 모델 구성

이제 노이즈가 포함된 이미지를 입력으로 받아 깨끗한 이미지로 복원하도록 학습할 오토인코더 모델을 구성합니다.

이번 예제에서는 완전 연결층 기반의 간단한 오토인코더를 구현합니다. 입력 이미지($28 \times 28 = 784$ 차원)를 잠재 공간에서 압축한 다음, 원래 이미지로 복원하는 모델입니다.

모델 설계에서 노드 수는 다음과 같습니다.

- 인코더에서는 $784 \rightarrow 128 \rightarrow 64 \rightarrow 32$ 로 압축
- 디코더에서는 $32 \rightarrow 64 \rightarrow 128 \rightarrow 784$ 로 늘어나게 구성

설계한 구조대로 다음과 같이 코드를 작성합니다.

CODE

```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        # Encoder: 784 → 128 → 64 → 32
        self.encoder = nn.Sequential( ①
            nn.Linear(28*28, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU() )
        # Decoder: 32 → 64 → 128 → 784
        self.decoder = nn.Sequential( ②
            nn.Linear(32, 64), nn.ReLU(),
            nn.Linear(64, 128), nn.ReLU(),
```

```

nn.Linear(128, 28*28),
nn.Sigmoid() ) # 픽셀 범위를 [0,1]로 맞추기 위해 시그모이드 함수 사용
def forward(self, x):
    x = x.view(x.size(0), -1) ③
    encoded = self.encoder(x) ④
    decoded = self.decoder(encoded) ⑤
    return decoded.view(x.size(0), 1, 28, 28) ⑥

```

- ① 입력 이미지의 크기를 $28 \times 28 = 784$ 차원 벡터로 펼친 다음(③), $128 \rightarrow 64 \rightarrow 32$ 차원으로 점진적으로 축소합니다. 층마다 ReLU 활성화 함수로 비선형성을 부여해 단순 선형 조합으로는 포착하기 어려운 패턴을 학습하고 압축 과정에서도 중요한 특징을 보존하도록 돕습니다. 인코더의 마지막 결과는 32차원 벡터입니다.
- ② 인코더가 만든 32차원 벡터를 다시 784차원으로 확장해 원래 이미지로 복원합니다. 마지막에 시그모이드 함수를 사용한 까닭은 출력값을 0~1 범위로 제한해 MNIST의 픽셀 범위와 일치시키기 위함입니다.
- ③ forward 메서드에서 입력으로 받은 2D 이미지(28×28)를 1차원 벡터(784)로 펼칩니다. `x.view(x.size(0), -1)`은 배치 차원(첫 번째 축)의 크기는 그대로 유지하고 나머지 차원을 모두 곱해 하나의 특징 차원으로 펼칩니다. 예를 들어 입력이 $[128, 1, 28, 28]$ 이라면 $[128, 784]$ 로 변환하며 이는 기능적으로 `nn.Flatten` 함수와 동일합니다.
- ④ 인코딩하며 32차원으로 압축합니다. 이렇게 압축된 32차원 벡터는 데이터를 요약해 담은 표현이기 때문에 잠재 벡터(Latent Vector)라고 합니다.
- ⑤ 디코더로 32차원을 다시 784차원으로 복원합니다.
- ⑥ 복원된 1차원 벡터를 `[batch_size, 1, 28, 28]` 형태로 다시 변형해 원래의 이미지 형태로 되돌립니다.

오토인코더 모델은 인코더에서 비선형 변환을 활용해 입력 정보의 손실을 최소화 하면서 압축하고, 디코더에서 같은 정보로 원본을 재구성하도록 학습합니다. ReLU 활성화 함수로 비선형 표현력을 확보하고, Sigmoid 함수로 출력값을 $[0, 1]$ 범위로 제한해 학습과 시각화 모두에서 안정적인 결과를 얻을 수 있습니다.

모델 선언 및 학습

이제 모델을 선언하고 손실 함수와 옵티마이저를 설정합니다. 오토인코더는 입력과 출력(예측값)이 최대한 같아지도록 학습합니다. 따라서 출력 이미지와 원본 이미지 간의 차이를 줄이는 것이 목표입니다. 차이를 줄이기 위해 가장 많이 사용하는 손실 함수는 평균 제곱 오차(Mean Squared Error, MSE)입니다. 그리고 옵티마이저로는 Adam을 사용합니다.

CODE

```
model = Autoencoder().to(device)
loss_fn = nn.MSELoss()
optimizer = Adam(model.parameters(), lr=0.001)
```

계속해서 준비된 모델과 손실 함수, 옵티마이저를 이용해 학습시킵니다. 노이즈를 제거하는 오토인코더 모델을 학습시킬 때는 입력에는 노이즈가 포함된 이미지를 넣고 출력에는 정상적인 이미지를 넣은 다음, 오차를 계산합니다.

다음과 같이 학습을 수행합니다.

CODE

```
epochs = 20
for t in range(epochs):
    model.train()
    running_loss = 0.0
    for imgs, _ in train_loader:
        noisy_imgs = add_noise(imgs).to(device) ①
        imgs = imgs.to(device) ②
        outputs = model(noisy_imgs) ③
        loss = loss_fn(outputs, imgs) ④
        optimizer.zero_grad() ⑤
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    tr_loss = running_loss / len(train_loader)
    print(f"Epoch {t+1}, train loss: {tr_loss:.6f}")
```

OUTPUT

```
[Epoch 1] Training Loss: 0.068906
[Epoch 2] Training Loss: 0.045545
[Epoch 3] Training Loss: 0.036148
...
[Epoch 18] Training Loss: 0.021408
[Epoch 19] Training Loss: 0.021106
[Epoch 20] Training Loss: 0.020792
```

- ① train_loader로 입력값(x)과 정답(y)을 불러오지만, x만 사용하고 y는 _로 처리합니다. 오토인코더의 학습은 y를 사용하지 않는 비지도학습입니다.
- ② add_noise 함수로 imgs에 노이즈를 추가하고 noisy_imgs에 저장합니다. 그리고 디바이스를 할당합니다.
- ③ imgs 역시 디바이스에 할당하는 이유는 모델의 입력으로 사용한 노이즈 데이터(noisy_imgs)의 출력 결과와 원본 데이터(imgs)를 비교하기 위함입니다.
- ④ 노이즈 이미지(noisy_imgs)를 오토인코더 모델에 입력해 복원 이미지를 출력합니다.
- ⑤ outputs는 모델이 복원한 이미지, imgs는 정답(깨끗한 원본 이미지)입니다. 두 이미지의 차이를 계산해 모델의 오차(MSELoss)를 구합니다.

오토인코더는 원래 정상 이미지를 입력으로 넣고 출력과 정상 이미지 사이의 오차를 계산합니다. 그러나 디노이징 오토인코더(Denoising Autoencoder)라는 분야가 발전하면서 노이즈가 포함된 이미지를 입력으로 학습시키면 모델이 원본 패턴을 복원하는 능력을 자연스럽게 익히게 되어 노이즈 제거 효과가 높습니다.

예측 및 결과 비교

학습된 모델로 예측을 수행합니다. 다음과 같이 테스트의 데이터 로더에서 배치를 하나를 뽑아 예측합니다.

CODE

```
model.eval() ①
imgs, _ = next(iter(test_loader)) ②
noisy_imgs = add_noise(imgs).to(device) ③
with torch.no_grad(): ④
    outputs = model(noisy_imgs) ⑤
```

- ① 모델을 평가 모드로 전환합니다.
- ② test_loader에서 첫 번째 배치를 꺼내 입력값(x)과 정답(y)을 가져옵니다. 정답(y)은 사용하지 않으므로 _로 무시하고 imgs만 활용합니다.
- ③ 불러온 이미지에 노이즈를 추가한 다음 디바이스를 할당합니다.
- ④ 평가 단계에서는 기울기(Gradient) 계산이 필요하지 않으므로 torch.no_grad 블록에서 실행합니다.
- ⑤ 노이즈가 포함된 이미지를 모델에 넣어 복원 이미지를 얻습니다. 오토인코더 모델의 학습에 따라 outputs에는 원본에 가까운 이미지를 재구성한 결과가 담기게 됩니다.

이제 원본, 노이즈 데이터, 복원된 결과(예측 결과) 이미지를 각각 6개씩 뽑아 비교합니다.

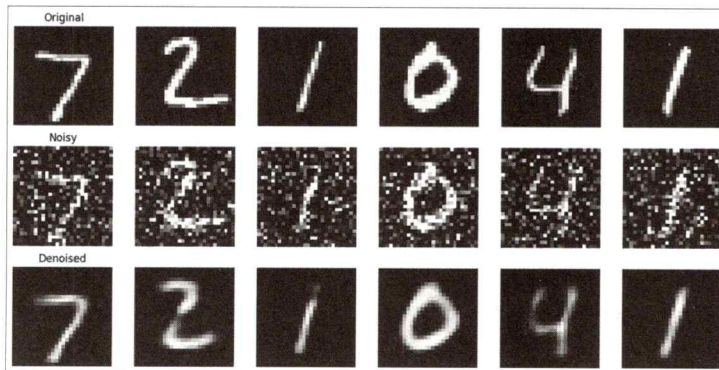
CODE

```
imgs = imgs[:6].cpu() ①
noisy_imgs = noisy_imgs[:6].cpu() ②
outputs = outputs[:6].cpu() ③
plt.figure(figsize=(12, 6))
for i in range(6):
    # 원본
    plt.subplot(3, 6, i + 1)
    plt.imshow(imgs[i][0], cmap='gray')
    plt.axis('off')
    if i == 0: plt.title("Original") ④
    # 노이즈
    plt.subplot(3, 6, i + 7)
```

```
plt.imshow(noisy_imgs[i][0], cmap='gray')
plt.axis('off')
if i == 0: plt.title("Noisy")
# 복원
plt.subplot(3, 6, i + 13)
plt.imshow(outputs[i][0], cmap='gray')
plt.axis('off')
if i == 0: plt.title("Denoised")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT



- ① 테스트 배치에서 앞쪽 6개의 원본 이미지만 추출하고 GPU에 있다면 CPU로 옮깁니다.
- ② 노이즈가 추가된 이미지도 앞쪽 6개만 가져와 CPU로 옮깁니다.
- ③ 오토인코더가 복원한 결과 이미지 역시 6개만 선택해 CPU로 옮깁니다.
- ④ 첫 번째 이미지(인덱스 0)에만 제목을 붙입니다. 따라서 각 열의 첫 번째 그림 위에 'Original', 'Noisy', 'Denoised'라는 제목이 표시됩니다.

코드 실행 결과는 첫 번째 행은 원본 이미지, 두 번째 행은 노이즈 이미지, 마지막 행은 오토인코더 모델 학습으로 노이즈가 제거된 이미지를 각각 출력하고 있습니다. 노이즈를 제거한 이미지의 결과를 보면 원본에 비해 해상도가 조금 떨어지지만, 노이즈를 상당히 제거하고 있음을 알 수 있습니다.

오토인코더 구현 2: MNIST 차원 축소

이번에는 오토인코더 정가운데 층(잠재 공간)의 노드를 2개로 구성한 다음, MNIST 원본 이미지를 입력해 압축하고 다시 원본 이미지를 복원하도록 모델을 학습시켰습니다. 그런 다음 모델에서 인코더 부분만 떼어 내면 MNIST의 고차원 데이터

(784차원)를 2차원 잠재 벡터로 변환할 수 있습니다. 그리고 잠재 벡터에 담긴 정보에 정답(숫자 범주)을 붙여 2차원 그래프로 시각화합니다. 이번 실습은 '10일차_비지도학습모델링.ipynb' 파일에서 이어 작성하면 됩니다.

모델 구성

모델을 학습한 다음 인코더만 따로 떼어 사용할 예정이므로 모델 클래스의 인코더를 별도의 함수로 구성합니다. 모델 클래스의 코드를 보면 인코더의 마지막 층 출력 노드 수와 디코더 첫 번째 층의 입력 노드 수가 2로 같다는 것을 알 수 있습니다. 그리고 클래스의 마지막 부분에 잠재 벡터 추출을 위한 인코더 함수를 별도로 정의했습니다.

```
CODE
class Autoencoder2D(nn.Module):
    def __init__(self):
        super(Autoencoder2D, self).__init__()
        self.encoder = nn.Sequential( ①
            nn.Linear(784, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 2)) ②
        self.decoder = nn.Sequential( ③
            nn.Linear(2, 64), nn.ReLU(),
            nn.Linear(64, 128), nn.ReLU(),
            nn.Linear(128, 784),
            nn.Sigmoid())
    def forward(self, x):
        x = x.view(x.size(0), -1)
        z = self.encoder(x)
        out = self.decoder(z)
        return out.view(x.size(0), 1, 28, 28)
    def encode(self, x): ④
        x = x.view(x.size(0), -1)
        return self.encoder(x)
```

① 인코더: $784 \rightarrow 128 \rightarrow 64 \rightarrow 2$

② 인코더의 마지막 층은 2차원입니다. 이렇게 만든 이유는 2차원 좌표에 시각화하기 위함입니다.

③ 디코더: $2 \rightarrow 64 \rightarrow 128 \rightarrow 784$

④ 잠재 벡터만 추출하기 위해 모델의 인코더만 별도의 메서드로 정의합니다. `encode` 메서드를 보면 입력 데이터를 펼쳐 `self.encoder`에 넣고 결과를 반환합니다.

784차원을 2차원으로 압축하고 2차원 잠재 벡터를 시각화하기 위한 모델을 구성했

습니다. 이 코드의 두 가지 특징은 2차원으로 압축한다는 점, 인코더 부분만 떼어 사용할 수 있도록 메서드를 추가한 점입니다.

모델 선언 및 학습

모델을 선언하고 학습을 시키는 과정은 앞의 오토인코더의 학습과 동일합니다. 다만 모델을 입력할 때 원본 데이터가 들어가는 부분만 신경 쓰면 됩니다.

CODE

```
model = Autoencoder2D().to(device)
loss_fn = nn.MSELoss()
optimizer = Adam(model.parameters(), lr=0.001)

epochs = 20
for t in range(epochs):
    model.train() # 학습 모드
    running_loss = 0.0
    for imgs, _ in train_loader:
        imgs = imgs.to(device)
        outputs = model(imgs) ①
        loss = loss_fn(outputs, imgs) ②
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    tr_loss = running_loss / len(train_loader)
    print(f"Epoch {t+1}, train loss: {tr_loss:.6f}")
```

OUTPUT

```
[Epoch 1] Training Loss: 0.067704
[Epoch 2] Training Loss: 0.053347
[Epoch 3] Training Loss: 0.049070
...
[Epoch 18] Training Loss: 0.041100
[Epoch 19] Training Loss: 0.040926
[Epoch 20] Training Loss: 0.040811
```

- ① 노이즈 제거를 위한 오토인코더에서는 노이즈가 포함된 이미지를 넣었지만, 이번에는 차원을 2차원으로 축소하고 시각화하는 게 목적이므로 모델 입력으로 원본 이미지를 사용합니다.
- ② 원본 이미지를 복원한 결과(outputs)와 원본 이미지(imgs) 사이의 오차를 계산합니다.

입력 이미지를 2차원 잠재 공간으로 압축하고 다시 복원하면서 오토인코더 모델을 학습시켰습니다.

모델 사용: 차원 축소

이제 테스트 데이터의 데이터 로더에서 배치 단위로 뽑아 잠재 벡터를 만듭니다.

CODE

```
model.eval()
latent_vectors = []
labels = []
with torch.no_grad():
    for imgs, lbls in test_loader:
        imgs = imgs.to(device)
        z = model.encode(imgs) ①
        latent_vectors.append(z.cpu()) ②
        labels.append(lbls) ③
latent_vectors = torch.cat(latent_vectors) ④
labels = torch.cat(labels)
```

- ① 오토인코더 모델에서 encode 메서드를 사용해 입력 이미지를 2차원 잠재 공간으로 변환합니다. z의 크기는 [배치 크기, 2]로서 숫자 이미지 하나당 (x, y)의 좌표 형태(2차원)로 표현됩니다.
- ② 모델이 뽑아낸 잠재 벡터 z는 GPU 위에 있을 수 있습니다. 그런데 여러 배치를 모아서 하나로 합치려면 CPU에 있는 텐서가 더 편합니다. CPU로 옮기고 리스트 latent_vectors에 하나씩 추가합니다. 리스트에 추가하는 이유는 그래프를 손쉽게 그리기 위함입니다.
- ③ 각각의 이미지가 어떤 숫자인지 알려 주는 레이블(lbls)도 함께 모아 둡니다. 이렇게 하면 이후 잠재 공간을 그래프로 그릴 때 숫자가 같은 것끼리 색깔을 각각 다르게 표시할 수 있습니다.
- ④ 배치마다 분리되어 있던 텐서들을 하나의 큰 텐서로 합칩니다. 최종적으로 latent_vectors는 [전체 데이터 개수, 2] 크기의 텐서, labels는 [전체 데이터 개수] 크기의 텐서가 됩니다.

이 과정은 오토인코더가 학습한 잠재 공간을 실제로 추출하는 단계로 숫자 이미지가 어떻게 2차원 좌표 위에 배치되는지 확인할 수 있습니다. 잠재 벡터와 레이블을 함께 모아 두면 이후 시각화에서 데이터의 구조와 패턴을 한눈에 파악할 수 있습니다.

잠재 벡터 시각화

이제 앞에서 저장한 잠재 벡터 리스트(latent_vectors)와 레이블 리스트(labels)를 시각화합니다. 784차원을 2차원으로 축소한 잠재 벡터를 산점도(scatter)로 그리고 레이블별로 색을 달리해 시각화합니다.

CODE

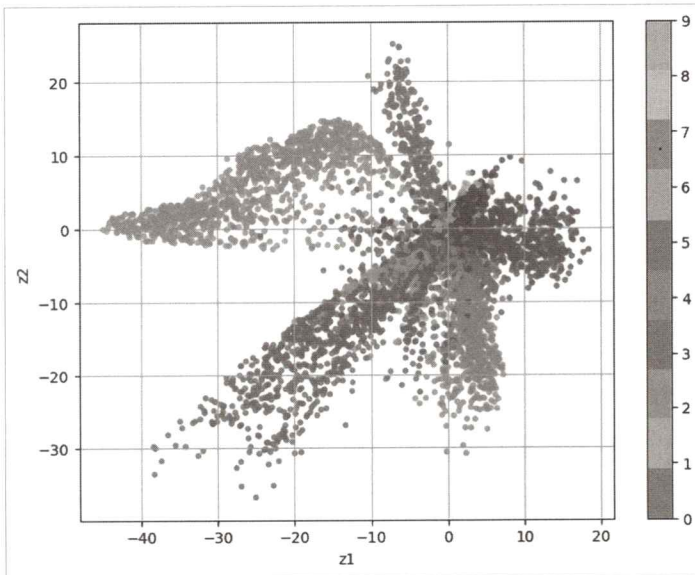
```
plt.figure(figsize=(8, 6))
scatter = plt.scatter(latent_vectors[:, 0], latent_vectors[:, 1], c=labels, ①
                    cmap='tab10', s=10, alpha=0.7) ②
plt.colorbar(scatter, ticks=range(10)) ③
```



```
plt.xlabel("z1") ④
plt.ylabel("z2")
plt.grid() ⑤
plt.show()
```

- ① latent_vectors의 첫 번째 열을 x좌표(z1), 두 번째 열을 y좌표(z2)로 사용해 산점도를 그립니다. c=labels로 지정했기 때문에 각각의 점은 숫자 레이블(0~9)에 따라 다른 색으로 표시됩니다.
- ② 색상표로 tab10을 사용해 10개의 고유 색상을 적용합니다. s=10으로 점의 크기, alpha=0.7로 투명도를 지정합니다. 점들이 겹쳐도 구분이 잘 되도록 하기 위함입니다.
- ③ 색상과 숫자 레이블이 어떻게 매칭되는지 확인할 수 있도록 색상 막대를 추가합니다. ticks=range(10)으로 지정해 숫자 0~9 레이블이 색상과 대응하도록 지정합니다.
- ④ x축의 이름을 z1로 지정합니다. 이는 잠재 공간의 첫 번째 차원을 의미합니다.
- ⑤ 그래프 위에 격자선을 표시해 점들의 분포를 좀 더 쉽게 확인할 수 있도록 합니다.

코드를 실행하면 [그림 10-4]와 같이 숫자 이미지별로 군집을 형성하는 그래프를 볼 수 있습니다. 딥러닝 학습에서 초기 가중치가 랜덤하게 할당되기 때문에 여러분이 그린 그래프와 책의 그래프는 형태가 조금 다를 수 있습니다.



[그림 10-4] 784차원 데이터를 2차원으로 축소한 후 시각화

숫자 0, 1, 2, 6은 2차원 잠재 벡터에서 잘 분리가 된 것처럼 보입니다. 그러나 9와 4 그리고 7은 겹친 부분이 꽤 있습니다. 또한 3, 5, 8도 중복 영역이 존재합니다. 중복

영역이 존재하는 까닭은 MNIST 데이터에 있는 이 숫자들이 실제로 비슷해 보이거나 헛갈리게 작성한 손글씨이기 때문입니다.

잠재 공간으로 축소된 데이터(은닉층 데이터)의 의미를 정확히 파악할 수는 없지만, 잠재 벡터가 0~9까지의 손글씨를 분류하는 데 상당히 중요하고 의미 있는 값을 보여 주고 있습니다.

오토인코더의 인코더와 PCA

오토인코더의 인코더는 입력 데이터를 더 낮은 차원의 공간으로 압축하는 역할을 합니다. 이처럼 복잡한 데이터를 간단하게 요약하는 과정은 차원 축소와 매우 유사합니다. 그렇다면 차원 축소 기법으로 자주 사용되는 PCA(주성분 분석)와 오토인코더는 어떤 차이가 있을까요?

먼저 PCA는 데이터를 가장 잘 설명할 수 있는 방향(주성분)을 찾아서 데이터를 그 방향을 기준으로 정렬합니다. 예를 들어 3차원 공간의 점들을 2차원 평면 위에 그려야 할 때 PCA는 가장 정보가 많이 담긴 두 방향만 남기고 나머지는 버립니다. 이 과정을 통해 데이터를 손실 없이 최대한 간단하게 표현할 수 있습니다.

하지만 PCA는 어디까지나 직선을 기반으로 하는 선형 기법입니다. 따라서 데이터가 직선처럼 잘 퍼져 있는 경우에는 유효하지만, 나선형이나 곡선처럼 복잡한 구조를 가진 데이터에서는 한계가 있습니다.

반면 오토인코더는 신경망을 활용하기 때문에 훨씬 더 유연하게 작동합니다. 인코더는 비선형 함수와 여러 계층으로 구성되어 있기 때문에 복잡한 패턴이나 곡선 형태의 데이터 구조도 학습할 수 있습니다. 또한 구조를 다양하게 바꿀 수 있는 활성화 함수 등을 조절하기 때문에 더 복잡한 데이터에서도 잘 적용할 수 있습니다.

두 방법의 차이를 [표 10-2]로 정리하면 다음과 같습니다.

[표 10-2] PCA와 오토인코더의 차원 축소 비교

항목	PCA	오토인코더
방식	선형(직선 기반)	비선형도 가능(신경망 기반)
복잡한 패턴 처리	제한적	가능
계산 효율성	빠름	상대적으로 느림
구성 요소	수학적 계산 기반	신경망 구성(Encoder/Decoder)
활용 유연성	차원 축소, 시각화에 주로 사용	노이즈 제거, 이상 탐지 등 다양하게 활용 가능

정리하면 PCA는 단순하고 빠르지만 구조가 단순한 데이터에 적합하며, 오토인코더는 복잡한 구조의 데이터까지도 유연하게 학습할 수 있는 강력한 도구입니다. 실제로 데이터가 복잡할수록 오토인코더가 더욱 효과적인 차원 축소 방법이 될 수 있습니다.

정리

마지막 차시에서는 비지도학습의 기본 개념을 시작으로 대표적인 기법들과 오토인코더의 구조, 활용까지 함께 살펴보았습니다. 특히 오토인코더가 입력을 압축하고 다시 복원하는 과정을 통해 정답 없이도 데이터의 본질적인 특징을 효과적으로 학습할 수 있다는 점을 실습으로 체험했습니다. 노이즈가 있는 이미지에서 깨끗한 이미지를 복원하거나 데이터를 2차원 공간으로 표현해 시각화하는 과정은 오토인코더의 강력함을 잘 보여 줍니다.

이제 여러분은 지도학습뿐만 아니라, 비지도학습의 기본 원리와 활용 방법까지 이해하게 되었습니다. 이는 앞으로 생성 모델이나 고급 딥러닝 모델로 학습을 확장하는 데 든든한 발판이 될 것입니다. 이 책에서 다룬 내용을 바탕으로 여러분의 프로젝트와 지식 탐구를 계속 이어가기를 진심으로 바랍니다.